

NAME	KEN KIBUGI MUIRURI
TRACKING NUMBER	CS-SA12-26014
COHORT	CYBERSHUJAA COHORT 1-2026
TASK	USING THE METASPLOIT FRAMEWORK REPORT

Contents

1.0	Introduction	3
1.1	Introduction to Metasploit	3
1.2	Introduction to MFSconsole	4
2.0	MSF Components	4
2.1	Modules	4
2.2	Targets	6
2.3	Payloads	6
2.4	Encoders	8
2.5	Databases	8
2.6	Plugins & Mixins	8
3.0	MSF Sessions	9
3.1	Sessions & Jobs	9
3.2	Meterpreter	12
4.0	Additional Features	15
4.1	Writing & Importing Modules	15
4.2	Introduction to MSFVenom	15
4.3	Firewall and IDS/IPS evasion	15
4.4	Metasploit-Framework Updates – August 2020	15
	Conclusion	16

USING THE METASPLOIT FRAMEWORK REPORT

Introduction

In this module, I learned how to use the Metasploit Framework to identify vulnerabilities, generate and deliver payloads, and manage post-exploitation more efficiently. It helped me understand how real-world attacks are structured and how different tools within Metasploit work together during an assessment.

Here is a shareable link to my completed module:

<https://academy.hackthebox.com/achievement/2380391/39>

Introduction

1.1 Introduction to Metasploit

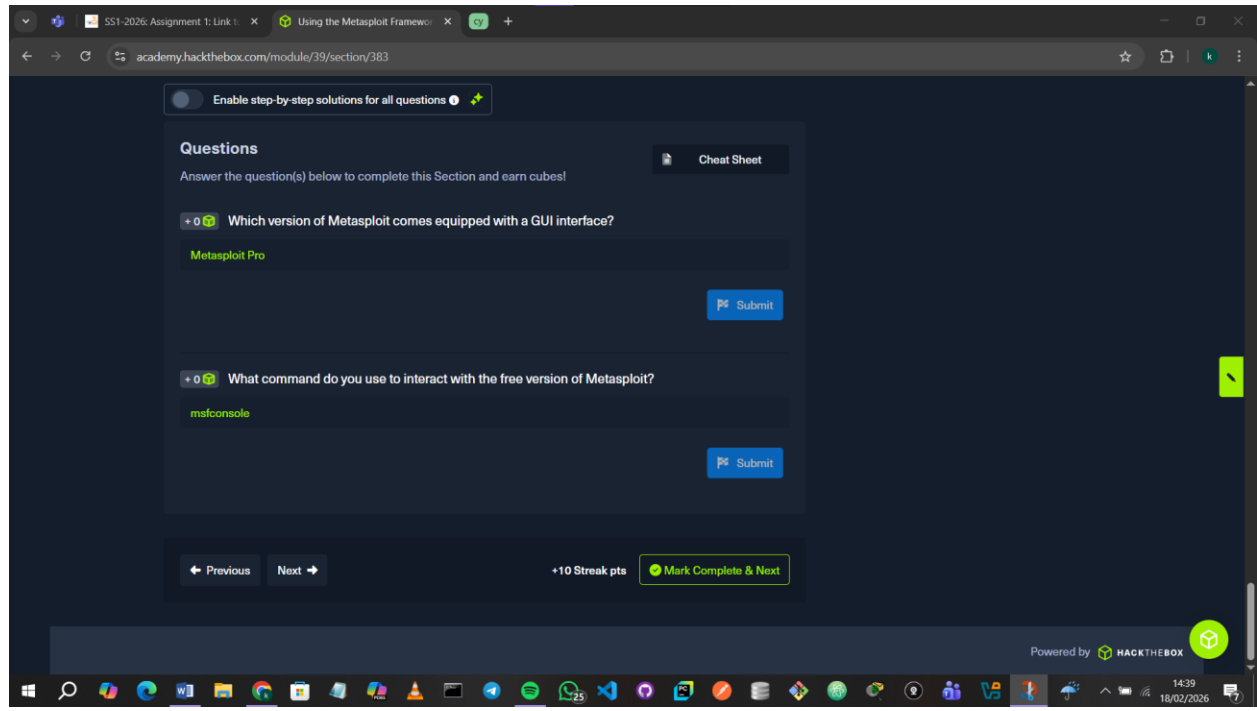
I learned that Metasploit is a modular penetration testing framework used to develop, test, and run exploits against known vulnerabilities. I also learned that **msfconsole** is the main interface, and understanding the framework's modules and file structure makes it easier to use Metasploit effectively during security assessments.

Question1: Which version of Metasploit comes equipped with a GUI interface?

Answer: **Metasploit Pro**

Question2: What command do you use to interact with the free version of Metasploit?

Answer: **msfconsole**



1.2 Introduction to MFSconsole

I learned that **msfconsole** is the main command-line interface used to interact with the Metasploit Framework. It's where I manage modules, run commands, and carry out the full attack workflow, starting from enumeration through exploitation and post-exploitation. I also learned that understanding the engagement structure (enumeration, exploitation, privilege escalation, and post-exploitation) makes using Metasploit more organized and effective.

2.0 MSF Components

2.1 Modules

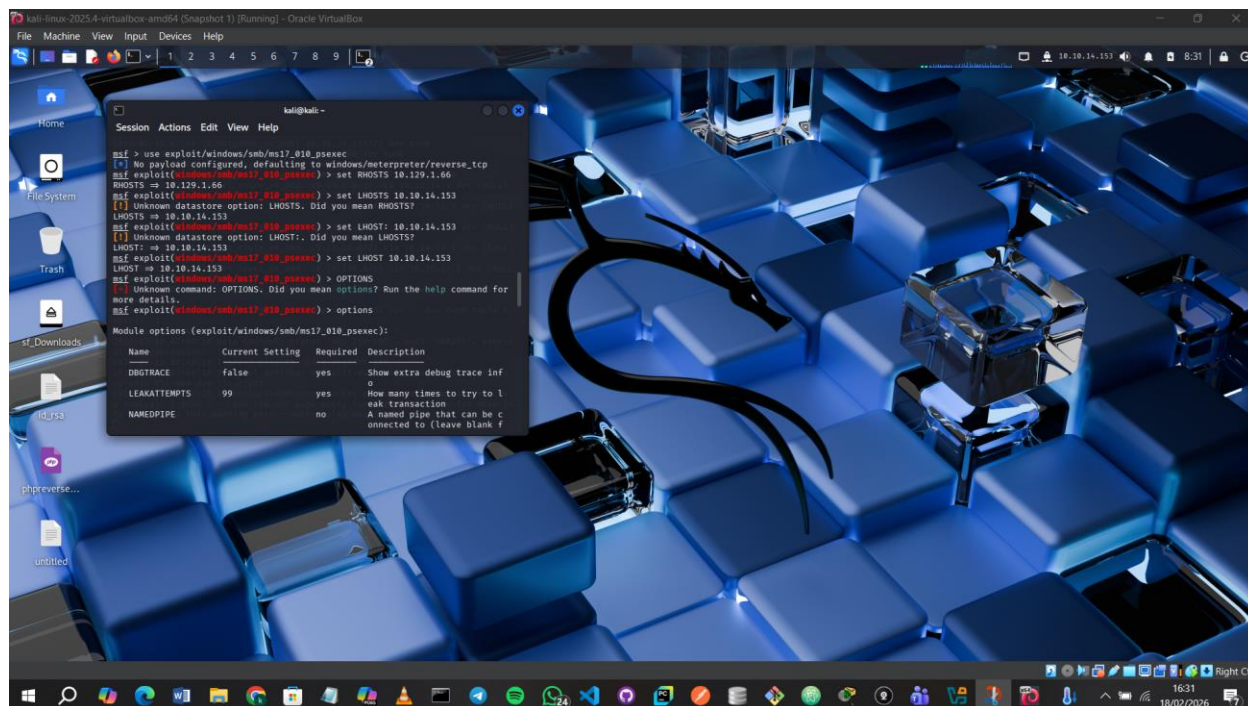
I learned that Metasploit modules are prebuilt, real-world tested scripts used for scanning, exploitation, payload delivery, and post-exploitation. I also learned how to search, select, and customize modules using options like RHOSTS and LHOST, and that failed exploits don't always mean a vulnerability doesn't exist—they often need tuning.

Question1: Use the Metasploit-Framework to exploit the target with EternalRomance. Find the flag.txt file on Administrator's desktop and submit the contents as the answer.

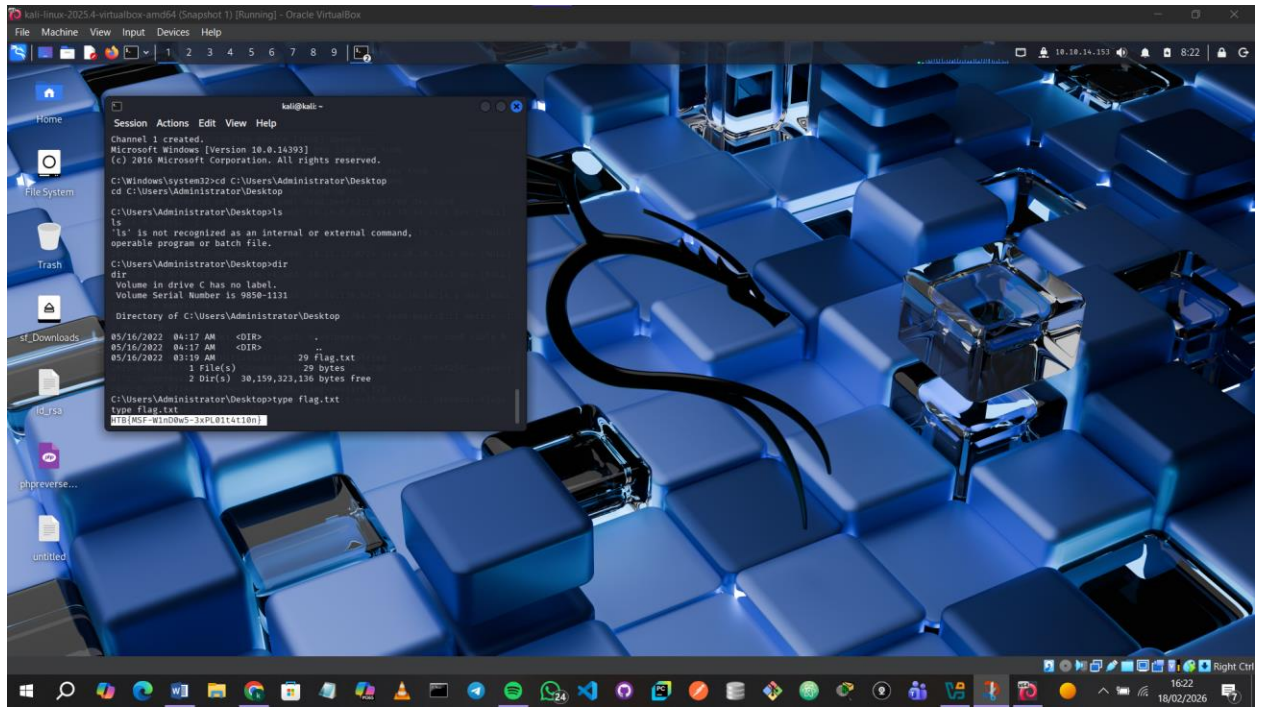
Answer: I started by launching Metasploit, finding the EternalRomance exploit by running **search ms17_010**.

I selected the exploit. Ran **use exploit/windows/smb/ms17_010_psexec**.

I then set RHOSTS and LHOST, then ran the exploit and landed in meterpreter.



Lastly I opened shell and got the flag. It was **HTB{MSF-W1nD0w5-3xPL01t4t10n}**



2.2 Targets

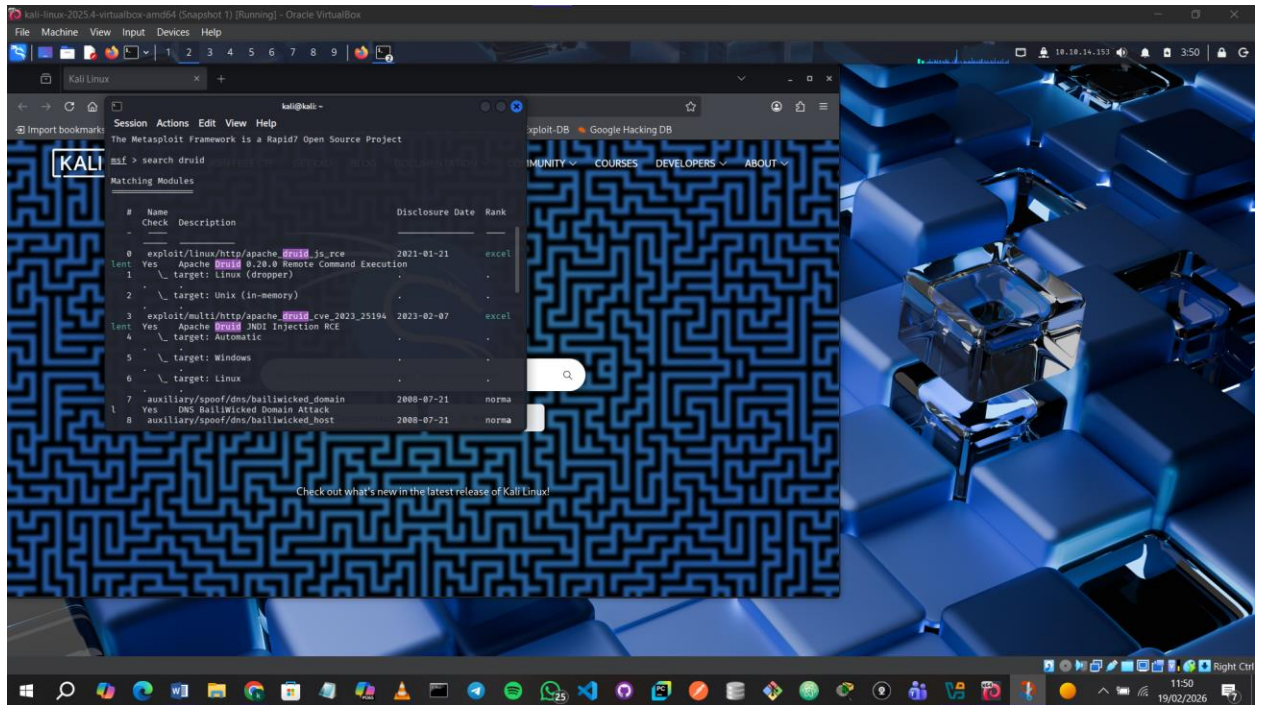
I learned that targets in Metasploit define the exact OS and software versions an exploit is built for. Using show targets, I can either let Metasploit auto-detect the target or manually select one if I already know the version. Correct target selection is crucial since exploits depend heavily on specific versions and memory addresses.

2.3 Payloads

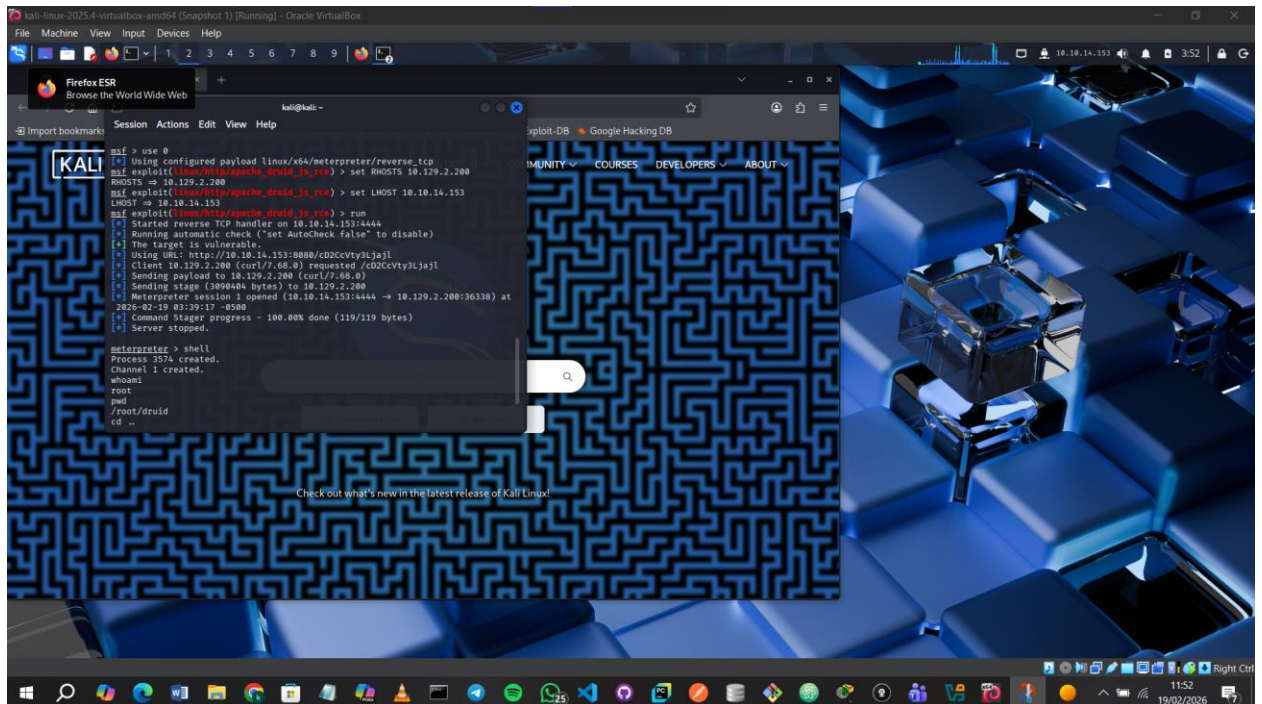
I learned that payloads work with exploits to give me access to a target system, usually by opening a reverse connection back to my machine. Metasploit payloads can be single, staged, or staged-with-stager, each balancing size, stability, and stealth. I also learned how powerful Meterpreter payloads are, since they run in memory and provide extensive post-exploitation capabilities.

Question: Exploit the Apache Druid service and find the flag.txt file. Submit the contents of this file as the answer.

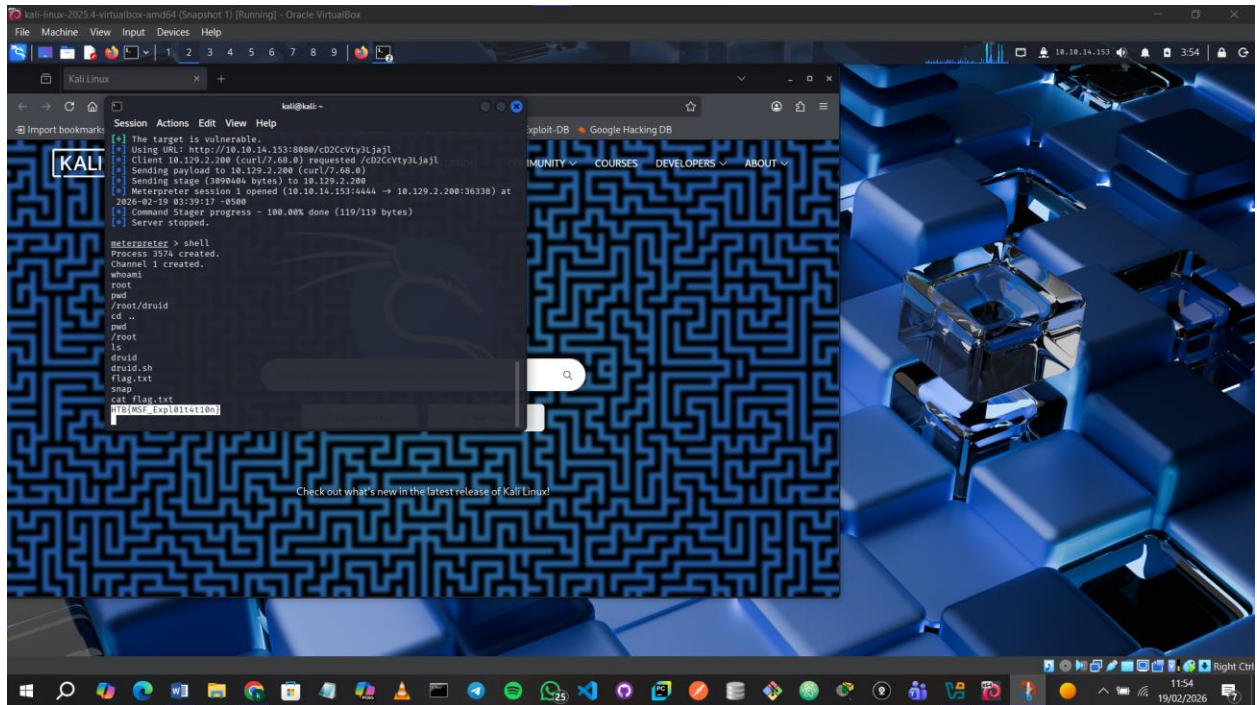
Answer: I started by launching msfconsole and ran search druid after which I ran use 0.



I then set RHOSTS and LHOST and was able to establish a session.



I opened shell and as root, I located the flag.txt file and opened it and found the flag as HTB{MSF_Exp10t4t10n}



2.4 Encoders

I learned that encoders modify payloads to match different architectures, remove bad characters, and slightly reduce antivirus detection. Older encoders like **Shikata Ga Nai** were once very effective, but modern AV solutions can now detect encoded payloads easily, even with multiple iterations. Today, encoders are mainly used for compatibility and payload formatting, not reliable AV evasion, meaning other techniques are required to bypass modern security systems.

2.5 Databases

I learned that databases in **msfconsole** help keep large assessments organized by storing hosts, services, vulnerabilities, credentials, and loot. Metasploit uses PostgreSQL to import scan results (like Nmap), run scans directly, and reuse findings when configuring exploits. Workspaces act like folders to separate different targets, and all collected data can be exported and backed up for later use.

2.6 Plugins & Mixins

I learned that Metasploit plugins extend **msfconsole** by integrating third-party tools directly into the framework. They help automate repetitive tasks, add new commands, and centralize results in the database, making assessments faster and more organized. Plugins can be loaded if already installed or added manually by placing their .rb files in the plugins directory, after which their features become immediately available inside msfconsole.

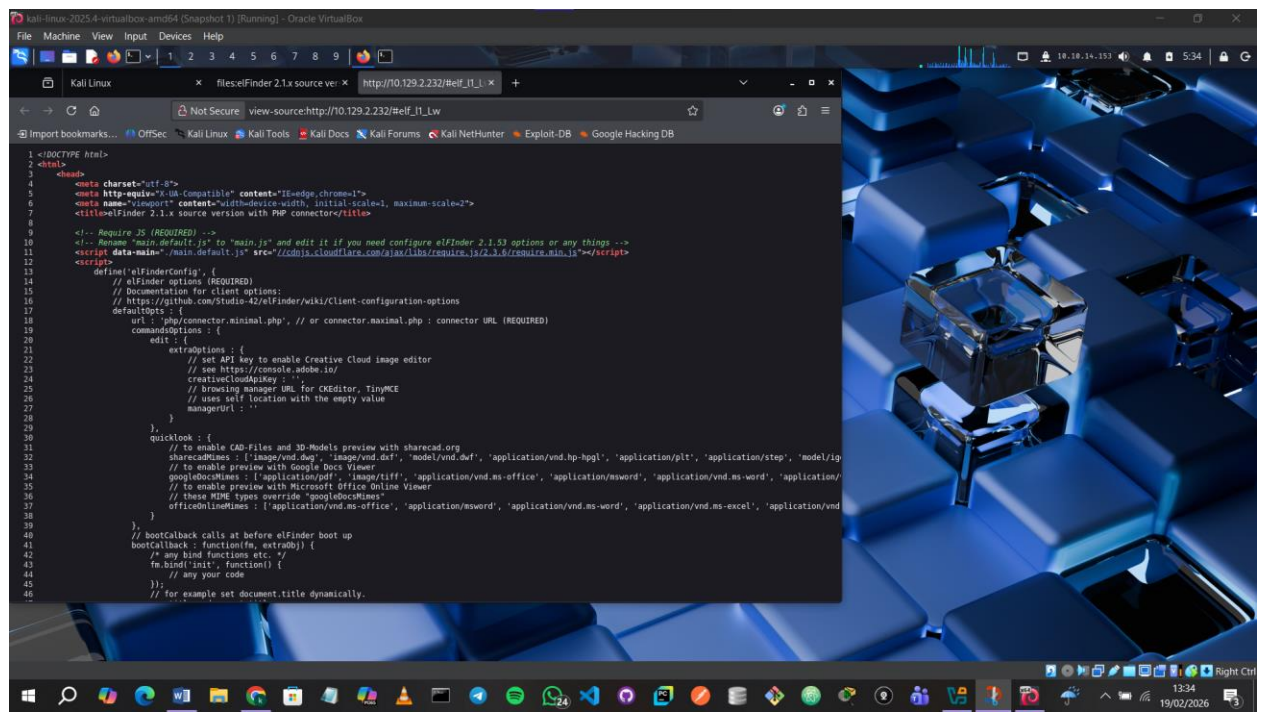
3.0 MSF Sessions

3.1 Sessions & Jobs

I learned that sessions let me manage multiple active connections in msfconsole and switch between them without losing access to targets. Jobs allow modules and exploits to run in the background, helping free up the console and manage ports efficiently during an assessment.

Question1: The target has a specific web application running that we can find by looking into the HTML source code. What is the name of that web application?

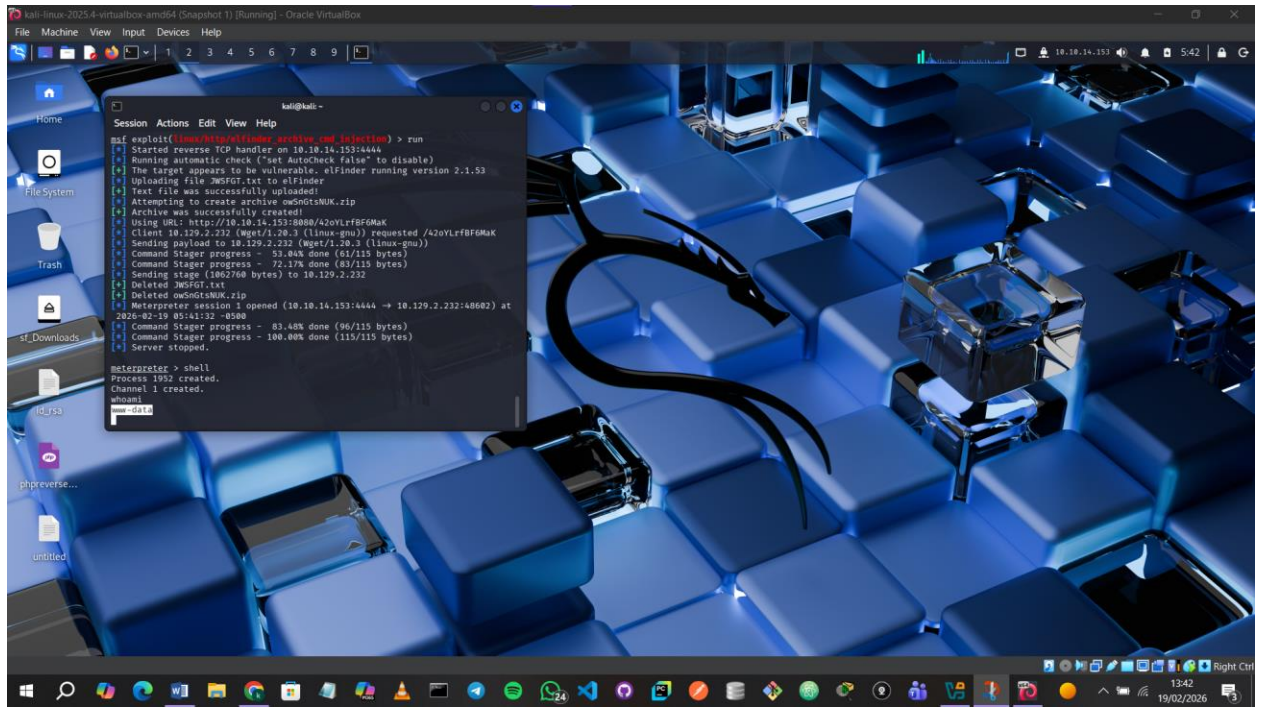
Answer: **elFinder**



Question2: Find the existing exploit in MSF and use it to get a shell on the target. What is the username of the user you obtained a shell with?

Answer: I launched msfconsole, ran **search elFinder**, found an exploit and started a session.

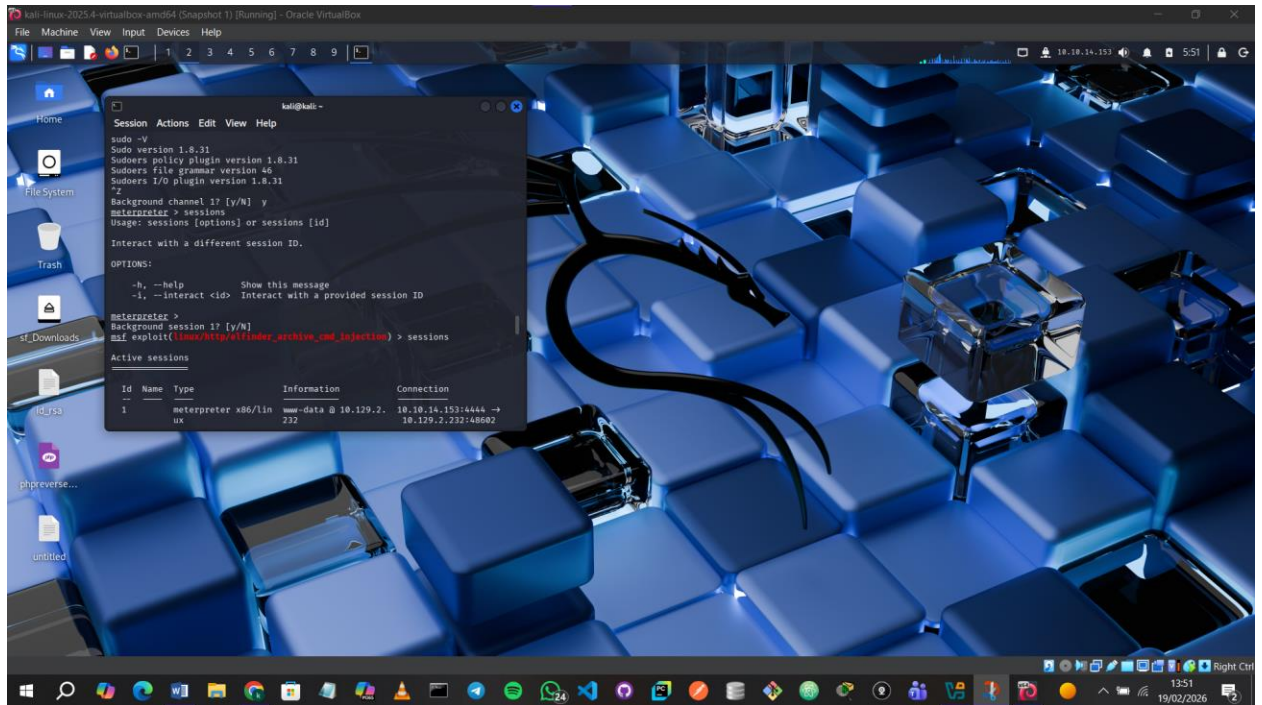
After successfully establishing a session I opened shell and found the username as **www-data**.



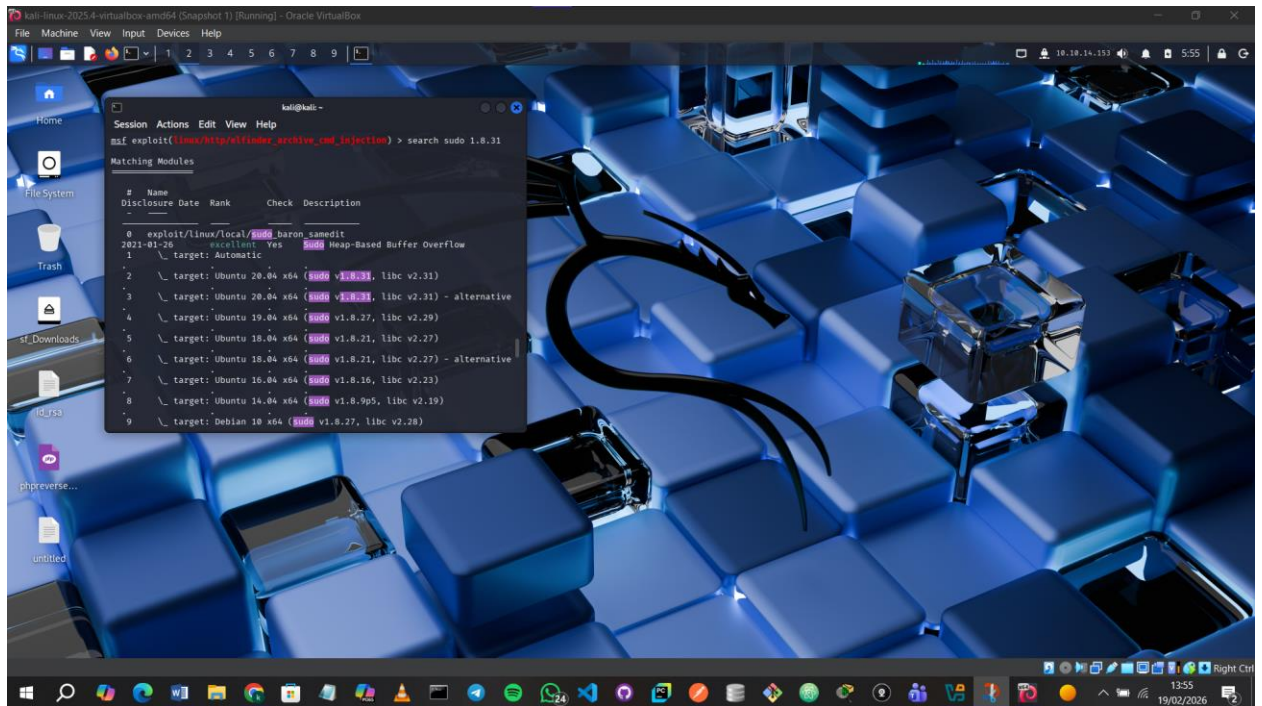
Question3: The target system has an old version of Sudo running. Find the relevant exploit and get root access to the target system. Find the flag.txt file and submit the contents of it as the answer.

Answer:

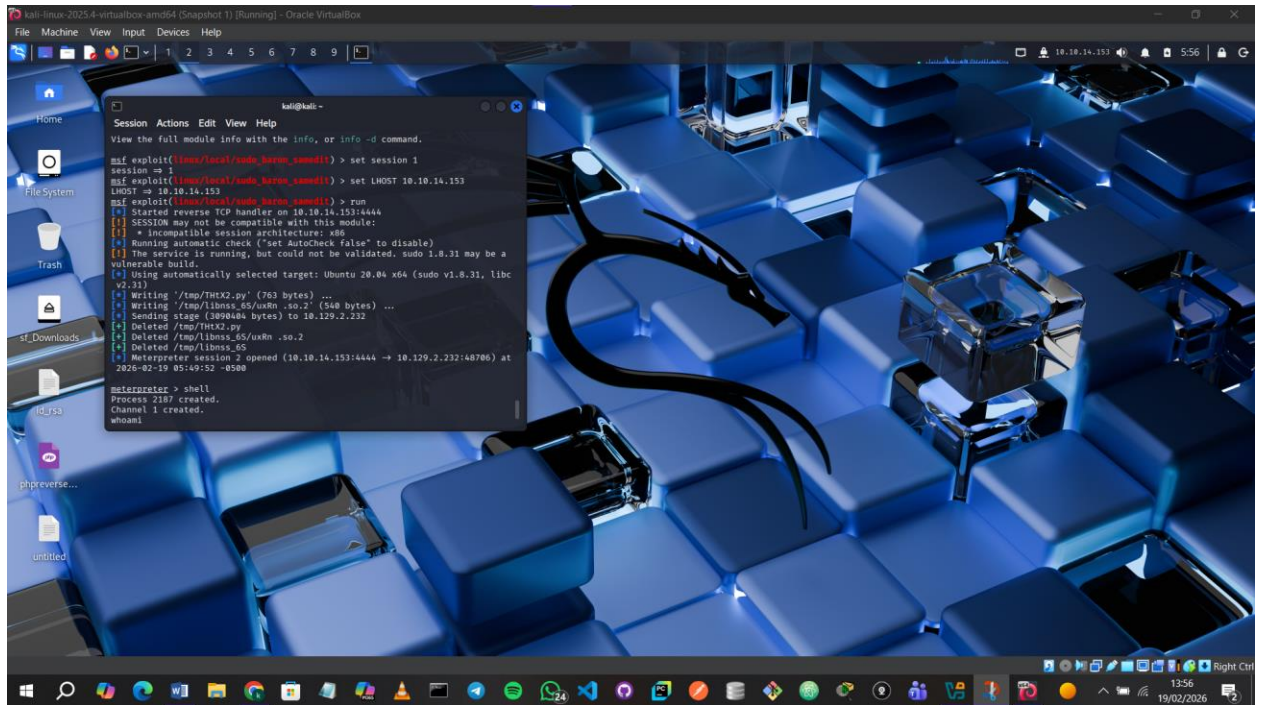
I started by finding the sudo version after which I backgrounded the session.



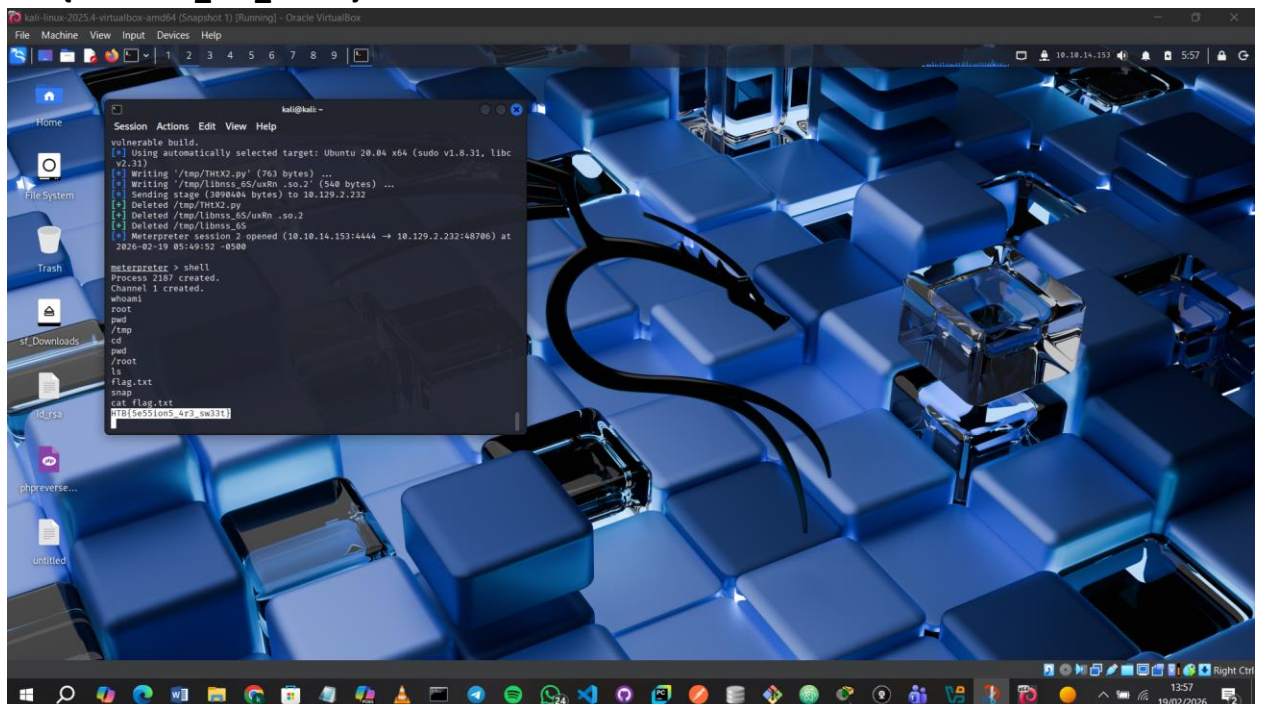
I then ran search sudo 1.8.31 and found an exploit which I used.



I set RHOSTS, LHOST and session and I was able to start a session.



I then opened shell and as root was able to locate the file and open it. The flag was **HTB{5e55ion5_4r3_sw33t}**



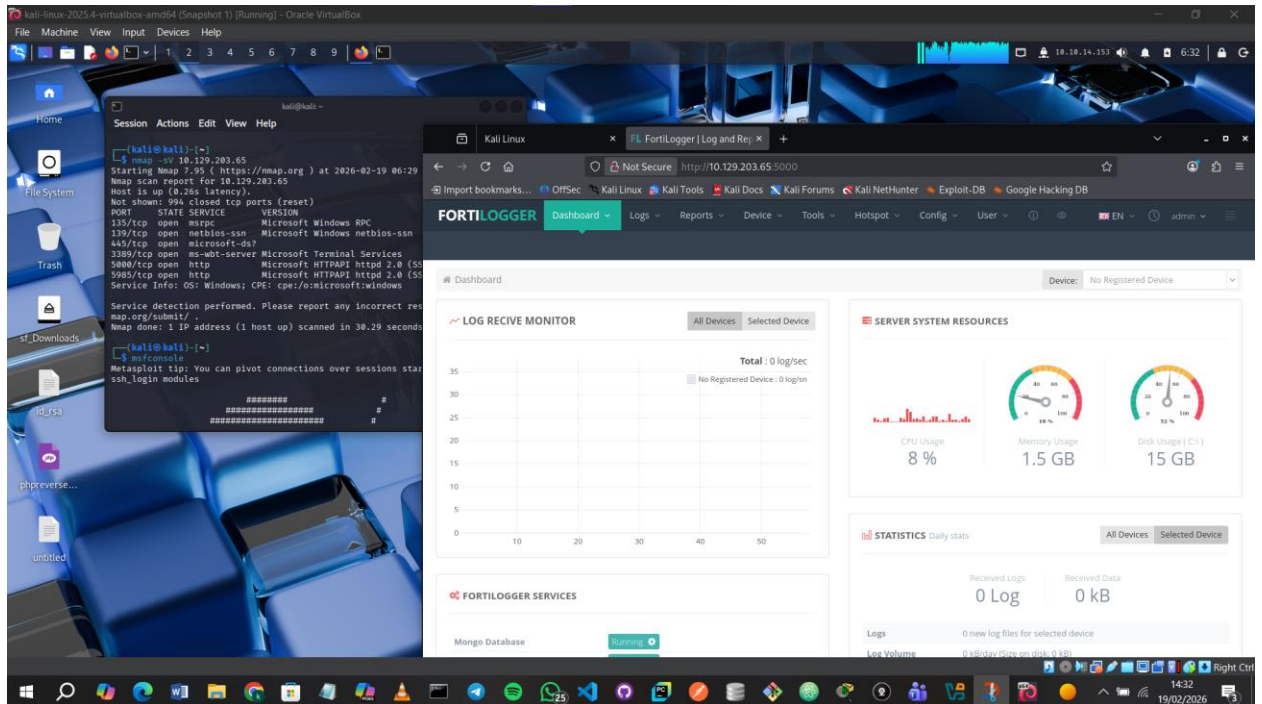
3.2 Meterpreter

I learned that Meterpreter is a powerful post-exploitation payload that runs entirely in memory, making it stealthy and hard to detect. It provides an interactive shell

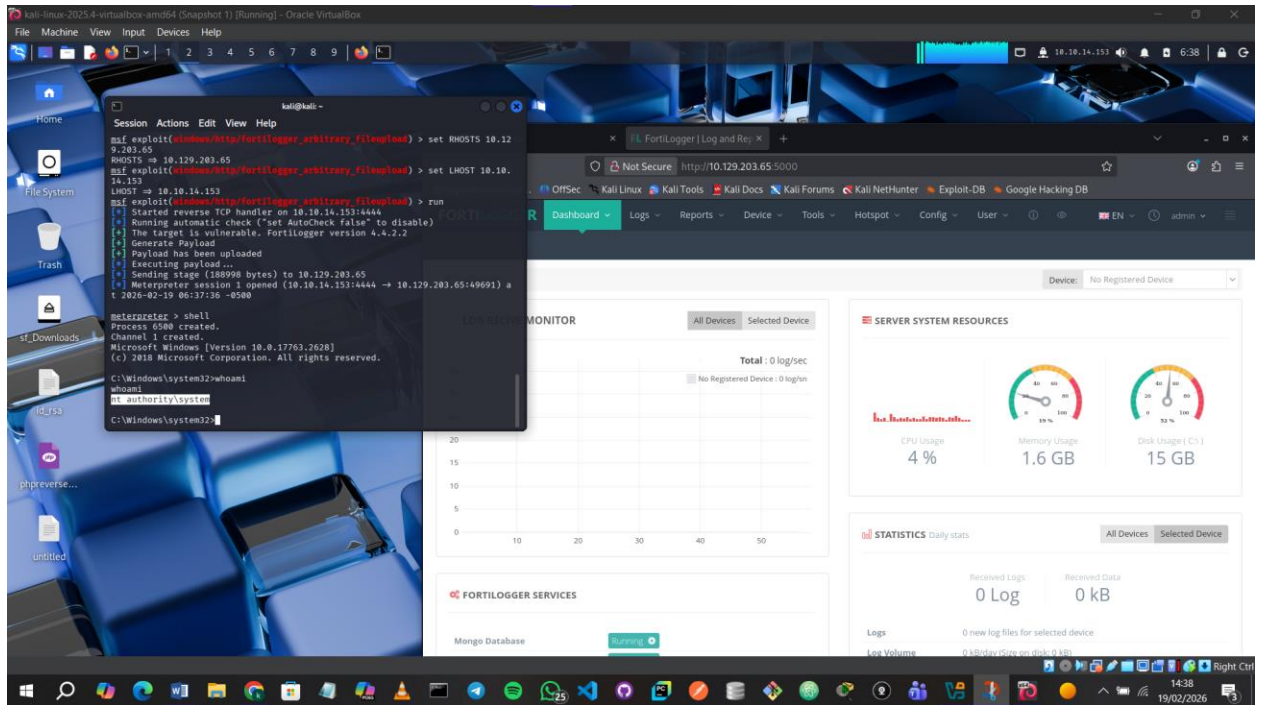
with advanced capabilities such as privilege escalation, process migration, credential dumping, and pivoting.

Question1: Find the existing exploit in MSF and use it to get a shell on the target.
What is the username of the user you obtained a shell with?

Answer: I ran an nmap scan and opened the address at port 5000.

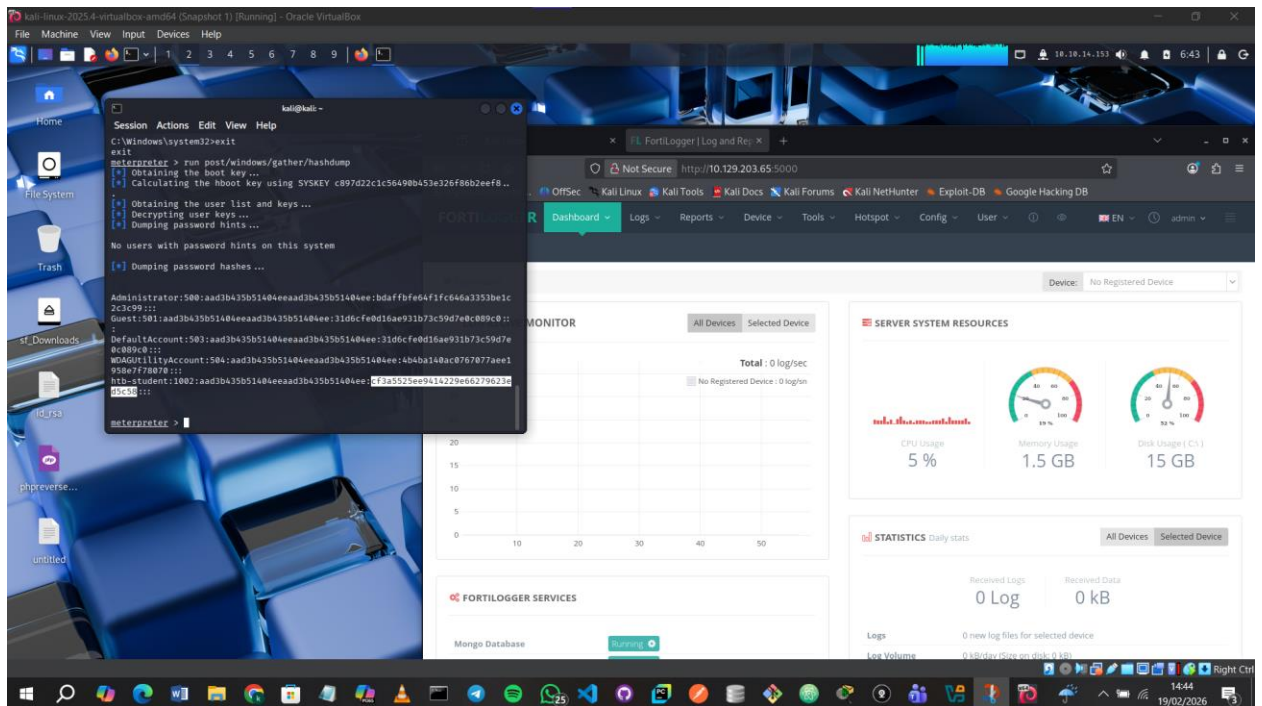


I then launched msfconsole and looked for any exploits for fortilogger, which I found and exploited. After successfully establishing a session, I opened shell and found the username as **nt authority\system**.



Question2: Retrieve the NTLM password hash for the "htb-student" user. Submit the hash as the answer.

Answer: I used the command **run post/windows/gather/hashtdump** to find it. The hash was **cf3a5525ee9414229e66279623ed5c58**.



4.0 Additional Features

4.1 Writing & Importing Modules

I learned that Metasploit modules can be updated automatically or manually imported from ExploitDB if they're not included by default. I also learned how to use searchsploit to find MSF-compatible exploits, place .rb files in the correct directories, reload Metasploit, and port custom exploits into full Metasploit modules using Ruby.

4.2 Introduction to MSFVenom

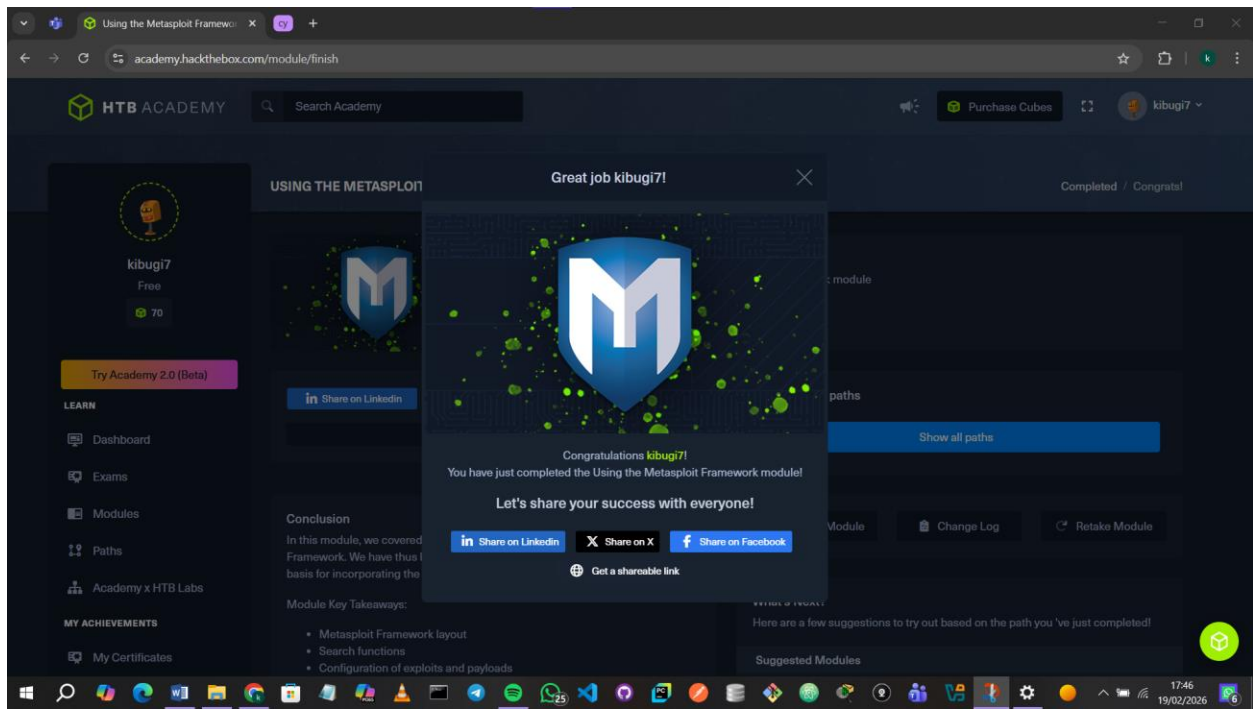
I learned that MSFvenom combines payload generation and encoding into a single tool, making it easier to create payloads tailored to specific target systems. It allows penetration testers to generate, format, and optionally encode payloads that can be used to gain initial access and establish reverse connections during exploitation.

4.3 Firewall and IDS/IPS evasion

I learned that evasion depends on understanding endpoint and perimeter defenses and how IDS/IPS and antivirus tools detect threats using signatures and behavior. Techniques like encrypting traffic, embedding payloads into legitimate files, using packers, and archiving payloads can significantly reduce detection and improve the chances of a successful attack.

4.4 Metasploit-Framework Updates – August 2020

I learned that upgrading to MSF6 introduced major changes, including breaking compatibility with MSF5 payloads and sessions. MSF6 added end-to-end AES encryption, polymorphic payload generation, cleaner payload artifacts, SMBv3 support, and replaced Mimikatz with Kiwi, all of which improve stealth, security, and overall exploitation workflows.



Conclusion

Through this module, I gained hands-on experience with Metasploit's core features, including payload generation, session handling, privilege escalation, and basic evasion techniques. Overall, it improved my understanding of how attackers leverage Metasploit during real-world exploitation and how such attacks can be carried out step by step.